

Exqutor 관련 References 6 편 상세 정리

선정 기준: Exqutor 논문을 깊이 이해하고 팀원에게 설명하기 위해, 논문의 핵심 기반 시스템 (pgvector 가 돌아가는 PostgreSQL 환경의 한계, VBASE, SingleStore-V), 문제의 출발점 (AnalyticDB-V), 이론적 배경(컨텍스트 강화 관계형 연산), 그리고 전문 벡터 DB 와의 대비 (Milvus)를 다루는 6 편을 선정했다.

논문 1. [1] AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data

저자: Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, Yuanzhe Cai (Alibaba Group)

학회/년도: VLDB 2020 (Proceedings of the VLDB Endowment, Vol. 13, No. 12)

분량: 약 14 페이지

이 논문의 의미

AnalyticDB-V 는 Exqutor 가 풀려는 문제, 즉 "벡터 검색과 SQL 분석 쿼리를 하나의 시스템 안에서 효율적으로 처리하는 것"을 **최초로 산업 규모에서 구현한 시스템**이다. Exqutor 논문의 Related Work 에서 직접 인용하며, VAQ(Vector-Augmented Analytical Query) 개념의 원형이 된다.

1.1 해결하고자 하는 문제

현대 데이터 파이프라인에서는 **구조화 데이터**(매출, 재고, 사용자 정보 등 숫자/텍스트 테이블)와 **비구조화 데이터**(이미지, 동영상, 오디오, 텍스트 문서 등)가 함께 생성된다.

기존에는 이 두 종류의 데이터를 별개의 시스템으로 처리했다:

- 구조화 데이터 → 관계형 데이터베이스 (MySQL, PostgreSQL 등)
- 비구조화 데이터 → 별도의 벡터 검색 엔진 (Faiss 등)

문제는 실제 비즈니스에서 두 데이터를 **함께** 분석해야 하는 경우가 매우 많다는 것이다. 예를 들어:

- "이 상품 이미지와 비슷한 상품을 찾되, 가격이 100 만원 이하이고 재고가 있는 것만 보여줘"
- "이 얼굴 사진과 유사한 사용자를 찾아서, 그 사용자의 최근 구매 이력을 분석해줘"

이런 쿼리를 처리하려면 개발자가 두 시스템 사이에서 수동으로 데이터를 옮기고 결합해야 했다. 이 과정이 매우 복잡하고 느렸다.

1.2 핵심 아이디어: SQL 로 하이브리드 쿼리 표현

AnalyticDB-V 의 핵심 제안은 **벡터 유사도 검색을 SQL 의 물리적 연산자(Physical Operator)로 구현**하는 것이다. 사용자는 하나의 SQL 문으로 벡터 검색과 관계형 분석을 동시에 표현할 수 있다.

```
SELECT product_name, price, similarity_score
FROM products
WHERE category = 'electronics'
      AND ANNS(image_embedding, query_vector, 10) -- 벡터 유사도 top-10
ORDER BY similarity_score DESC;
```

이렇게 하면 데이터베이스 옵티마이저가 벡터 검색과 필터링의 순서, 방식 등을 자동으로 결정해준다.

1.3 시스템 구조

AnalyticDB-V 는 **Lambda 아키텍처**를 채택한다:

스트리밍 레이어(Streaming Layer):

- 실시간으로 들어오는 벡터 데이터를 처리한다.
- 새로 삽입된 벡터는 즉시 검색 가능하다.
- 인메모리 임시 인덱스를 사용한다.

배치 레이어(Batching Layer):

- 주기적으로 새로 삽입된 벡터들을 압축하고, ANNS 인덱스를 재구축한다.
- 대규모 데이터에 대해 최적화된 영구 인덱스를 관리한다.

쿼리 처리 레이어:

- 스트리밍 레이어와 배치 레이어의 결과를 병합하여 최종 결과를 반환한다.

1.4 VGPQ (Vector Graph Product Quantization) 알고리즘

AnalyticDB-V의 핵심 기술적 기여 중 하나는 **VGPQ**라는 새로운 ANN 검색 알고리즘이다.

기존의 **IVF-PQ** (Inverted File with Product Quantization) 방식은:

1. 벡터 공간을 여러 클러스터(셀)로 나누고
2. 검색할 때 가까운 클러스터 몇 개만 방문하여 후보를 찾는다.

VGPQ는 여기에 **그래프 구조**를 추가한다:

3. 앵커(anchor) 중심점을 찾고
4. 그래프를 탐색하며 관련 서브셀(subcell)들을 효율적으로 선택하고
5. 후보 벡터들의 거리를 계산하여 결과를 반환한다.

이 방식은 대규모 벡터(수억~수십억 건)에서 기존 IVF-PQ보다 검색 정확도와 속도를 모두 향상시킨다.

1.5 정확도 인식 비용 기반 최적화 (Accuracy-Aware Cost-Based Optimization)

가장 중요한 기여는 **ANNS 연산자를 데이터베이스 옵티마이저에 통합한 것**이다.

일반적인 데이터베이스 옵티마이저는 각 연산의 비용(I/O, CPU 등)을 추정하여 최적의 실행 계획을 선택한다. AnalyticDB-V 는 여기에 **정확도(accuracy)**라는 새로운 차원을 추가한다.

ANN 검색은 본질적으로 근사적(approximate)이기 때문에, 파라미터 설정에 따라 정확도가 달라진다:

- 파라미터를 높이면 → 정확도 높아지지만 비용(시간) 증가
- 파라미터를 낮추면 → 비용 줄지만 정확도 저하

옵티마이저는 사용자가 요구하는 정확도 수준을 만족하면서, 가장 비용이 적은 실행 계획을 선택한다.

1.6 본 논문과의 관계

AnalyticDB-V는 "벡터 검색을 SQL에 통합하자"는 아이디어를 제시했지만, **카디널리티 추정** 문제를 깊이 다루지 않았다. 즉, 벡터 검색 결과가 몇 건이나 나올지를 정확히 예측하는 문제는 남아있었다. Exqutor는 바로 이 빈틈을 메우는 연구이다.

논문 2. [6] Context-Enhanced Relational Operators with Vector Embeddings

저자: Viktor Sanca, Manos Chatzakis, Anastasia Ailamaki (EPFL)

출처: arXiv:2312.01476 (원본 [5] ICDE 2023 비전 논문의 확장 기술 논문)

분량: 약 12 페이지 + 실험 부록

이 논문의 의미

이 논문은 [5]의 확장판으로, "벡터 임베딩을 관계형 연산에 어떻게 통합할 것인가"라는 이론적 기반을 제공한다. Exqutor 가 벡터 유사도 검색을 SQL 쿼리 내에서 최적화하는 것의 이론적 근거가 된다. 특히, **임베딩 연산자(E-Operator)**와 **컨텍스트 강화 조인(E-Join)**의 정의는 VAQ 최적화의 수학적 기초를 이해하는 데 필수적이다.

2.1 해결하고자 하는 문제

현대 데이터 분석에서는 **구조화 데이터**(숫자, 날짜 등)와 **컨텍스트 풍부한 데이터**(텍스트, 이미지, 오디오 등)를 함께 분석해야 한다. 기존 관계형 연산자(SELECT, JOIN 등)는 정확한 일치(exact match)나 범위 비교에만 적합하고, "의미적으로 유사한" 데이터를 찾는 데는 부적합하다.

예를 들어, 두 테이블에서 "비슷한 상품 설명"을 기준으로 조인하고 싶다면:

```
-- 이런 쿼리는 기존 SQL 로 표현 불가
SELECT * FROM table_a JOIN table_b
ON semantic_similarity(a.description, b.description) > 0.8
```

현재는 개발자가:

1. 두 테이블의 텍스트를 추출하고
2. 별도의 ML 파이프라인으로 임베딩을 생성하고
3. 벡터 유사도를 계산하고
4. 결과를 다시 원래 데이터와 합치는

복잡한 수작업을 해야 한다.

2.2 임베딩 연산자 (E-Operator)

논문은 임베딩을 관계형 대수(Relational Algebra)의 공식 연산자로 정의한다:

$$E\mu(R) = \{t \in R, t \rightarrow \mu(t)\}$$

쉽게 말하면:

- 입력: 관계(테이블) R 과 임베딩 모델 μ
- 출력: R 의 각 튜플(행)에 대해 임베딩 벡터를 추가한 새로운 관계

이 연산자의 중요한 성질:

1. **역연산 가능**: $E\mu^{-1}(E\mu(R)) = R$ (원래 데이터 복원 가능)
2. **관계형 대수와 호환**: 기존의 관계형 규칙(선택 푸시다운, 조인 재정렬 등)이 그대로 적용

2.3 컨텍스트 강화 조인 (E-Join)

핵심 기여: 벡터 유사도 기반의 새로운 조인 연산자 정의

$$R \text{ JOIN}_{E,\mu,\theta} S \Leftrightarrow \sigma_{\theta}(E_{\mu}(R) \times E_{\mu}(S)) \Leftrightarrow E_{\mu}(R) \text{ JOIN}_{\theta} E_{\mu}(S)$$

이 수식의 의미:

- 두 테이블 R 과 S 를 각각 임베딩하고
- 임베딩 벡터 간의 유사도(코사인 유사도 등)가 임계값 θ 를 넘는 쌍을 조인 결과로 반환

여러 동치 표현이 있기 때문에, 옵티마이저가 상황에 맞는 가장 효율적인 방식을 선택할 수 있다.

2.4 비용 모델과 논리적 최적화

Naive 비용 (최악의 경우):

$$\text{Cost}(R \text{ JOIN } S) = |R| \times |S| \times (A + M + C)$$

- A = 데이터 접근 비용
- M = 모델(임베딩) 계산 비용 ← 이것이 핵심 병목!
- C = 유사도 연산 비용

각 튜플 쌍마다 임베딩을 계산하면 $|R| \times |S|$ 번의 모델 호출이 필요하다.

프리페치(Prefetch) 최적화 — 핵심 통찰:

$$\text{Cost}(R \text{ JOIN } S) = |R| \times |S| \times (A + C) + (|R| + |S|) \times M$$

각 튜플의 임베딩은 한 번만 계산하면 된다. 두 테이블을 미리 임베딩해두면, 모델 호출 비용이 $O(|R| \times |S|)$ 에서 $O(|R| + |S|)$ 로 줄어든다.

이것은 단순해 보이지만, 실무에서 ML 모델 호출이 가장 비싼 연산이기 때문에 **12 배** 이상의 속도 향상을 가져온다.

2.5 물리적 최적화: 텐서 조인 (Tensor Join)

논문의 가장 혁신적인 물리적 최적화는 **Nested Loop Join** 을 **행렬 곱셈**으로 변환하는 것이다:

R의 임베딩을 행렬 **R** ($|R| \times \text{dim}$), S의 임베딩을 행렬 **S** ($\text{dim} \times |S|$)로 구성하면:

$$D = R \times S^T$$

결과 행렬 D의 (i,j) 원소가 R의 i 번째 튜플과 S의 j 번째 튜플 간의 유사도가 된다.

왜 이것이 빠른가?

- 행렬 곱셈은 수십 년간 최적화된 라이브러리(Intel MKL, BLAS 등)를 활용할 수 있다.
- CPU의 SIMD 명령어(AVX-512 등)로 한 번에 32 개 연산을 병렬 처리할 수 있다.
- 메모리 접근 패턴이 예측 가능해서 캐시 효율이 매우 높다.

실험 결과:

- 10k × 10k 조인: 텐서 방식이 NLJ 대비 **7.7 배** 빠름
- 100k × 100k 조인: **2.9 배** 빠름
- SIMD 적용 시 추가 **2 배** 향상
- 전체적으로 Naive 대비 **약 100 배** 속도 향상

2.6 인덱스 조인 vs. 스캔 조인 분석

논문은 벡터 인덱스(HNSW) 사용이 항상 좋은 것은 아님을 보여준다:

- **낮은 선택도** (결과가 적을 때): 텐서 스캔이 더 빠름 (인덱스 탐색 오버헤드 > 스캔 비용)
- **중간 선택도** (20~30%): 인덱스가 유리
- **높은 선택도** (결과가 많을 때): 다시 텐서 스캔이 유리 (어차피 대부분 방문해야 하므로)

이 결과는 Exqutor 의 "정확한 카디널리티가 있어야 최적의 접근 방식을 선택할 수 있다"는 주장을 강하게 뒷받침한다.

2.7 본 논문과의 관계

이 논문은 "벡터 임베딩을 관계형 대수에 공식적으로 통합"하는 이론적 기초를 제공한다. Exqutor 는 이 이론 위에서, "그렇다면 실행 계획을 세울 때 벡터 연산의 카디널리티를 어떻게 정확히 추정할 것인가?"라는 실용적 문제를 해결한다. 특히, 인덱스 조인 vs. 스캔 조인의 선택이 선택도에 따라 달라진다는 이 논문의 발견은, Exqutor 에서 정확한 카디널리티 추정이 왜 중요한지를 직접 보여준다.

논문 3. [7] SingleStore-V: An Integrated Vector Database System in SingleStore

저자: Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu 외 다수

학회/년도: VLDB 2024 (Proceedings of the VLDB Endowment, Vol. 17, No. 12)

분량: 약 14 페이지

이 논문의 의미

SingleStore-V 는 Exqutor 가 "Related Work"에서 직접 비교하는 범용 벡터 DB 의 쿼리 최적화 연구이다. Exqutor 는 "SingleStore 는 필터를 인덱스 스캔에 직접 통합하지만, 단순 필터 쿼리에만 집중하고 복잡한 조인 쿼리에는 부적합하다"고 지적한다. 이 논문을 이해하면 Exqutor 의 접근법이 왜 필요한지를 더 명확히 알 수 있다.

3.1 배경: SingleStore 란?

SingleStore(구 MemSQL)는 OLTP(트랜잭션)와 OLAP(분석) 모두를 지원하는 **분산 관계형 데이터베이스**이다. 실시간 데이터 처리와 분석을 하나의 시스템에서 가능하게 설계되었다.

SingleStore-V는 이 SingleStore에 벡터 검색 기능을 통합한 것으로, **범용 벡터 데이터베이스**에 해당한다. Milvus 같은 전문 벡터 DB와 달리, 기존 SQL 쿼리와 벡터 검색을 하나의 SQL 문에서 처리할 수 있다.

3.2 핵심 기술 1: Top() 연산자

SingleStore-V의 가장 중요한 기술적 기여는 **Top() 물리적 연산자**이다.

기존 방식에서 top-k 벡터 검색을 SQL로 표현하면:

```
SELECT * FROM products
ORDER BY embedding <-> query_vector
LIMIT 10;
```

이를 나이브하게 실행하면:

1. 모든 벡터와 query_vector의 거리를 계산하고 (Full Scan)
2. 결과를 정렬하고 (Sort)
3. 상위 10 개를 반환한다 (Limit)

SingleStore-V의 Top() 연산자는 이 과정을 **테이블 스캔 단계에서 직접** 수행한다:

4. 테이블 스캔 시 벡터 인덱스를 활용해 바로 top-k를 찾고
5. Sort 단계를 건너뛰다

이렇게 하면 ORDER BY + LIMIT 패턴을 인식해서, 불필요한 전체 정렬 없이 효율적으로 처리할 수 있다.

3.3 핵심 기술 2: 세그먼트 단위 인덱스 구조

SingleStore 의 저장 구조는 **세그먼트(segment)** 기반이다. 데이터가 쓰기 최적화된 행 저장소 (rowstore)에서 읽기 최적화된 열 저장소(columnstore)로 이동할 때, 불변(immutable) 세그먼트로 관리된다.

SingleStore-V 는 이 구조를 활용하여:

Per-Segment 벡터 인덱스:

- 각 불변 세그먼트마다 독립적인 벡터 인덱스를 구축한다.
- 세그먼트가 불변이므로, 인덱스도 한 번 구축하면 수정할 필요가 없다.
- 새 데이터가 추가되면 새 세그먼트에 새 인덱스가 생긴다.

Cross-Segment 검색:

- 쿼리 시 모든 관련 세그먼트의 인덱스를 병렬로 검색하고
- 각 세그먼트의 top-k 결과를 병합하여 전체 top-k 를 반환한다.
- 분산 환경에서 여러 노드의 세그먼트를 병렬 처리할 수 있어 확장성이 좋다.

3.4 핵심 기술 3: 플러그형 벡터 인덱스

SingleStore-V 는 벡터 인덱스를 **블랙박스**로 취급한다. 즉, 인덱스 내부 구현에 의존하지 않고, 표준 인터페이스만 사용한다. 이 덕분에:

- **Faiss** 라이브러리의 IVF_FLAT, IVF_PQ, IVF_PQFS 등
- **hnswlib** 라이브러리의 HNSW_FLAT, HNSW_PQ 등

다양한 인덱스를 플러그인처럼 교체할 수 있다. 사용자는 데이터 특성에 맞는 최적의 인덱스를 선택하면 된다.

3.5 성능 결과

VectorDBBench 기준으로:

- **Milvus(전문 벡터 DB)**와 비슷한 수준의 벡터 검색 성능 달성
- **pgvector** 를 크게 상회하는 성능
- 양자화(quantization) 기반, 그래프 기반 인덱스 모두에서 일관된 성능

3.6 본 논문과의 관계

SingleStore-V의 최적화는 주로 **단일 테이블 내 top-k 검색을 효율화**하는 데 집중한다. Top() 연산자는 "벡터 검색 결과를 빨리 가져오기"에 특화되어 있다.

반면 Exqutor는 **여러 테이블에 걸친 조인 순서, 조인 방식, 전체 실행 계획 최적화**에 집중한다. 벡터 검색 자체를 빠르게 하는 것이 아니라, 벡터 검색의 결과 크기를 정확히 예측해서 옵티마이저가 더 나은 계획을 세우도록 돕는 것이다. 따라서 두 접근법은 **상호 보완적**이다.

논문 4. [10] VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity

저자: Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie 외 다수 (Microsoft Research Asia)

학회/년도: OSDI 2023 (USENIX Symposium on Operating Systems Design and Implementation)

분량: 약 19 페이지

이 논문의 의미

VBASE 는 Exqutor 가 직접 통합한 3 개 시스템 중 하나이며, Exqutor 논문의 실험에서 **가장 극적인 성능 향상(최대 10,000 배)**을 보인 시스템이다. VBASE 의 독특한 아키텍처를 이해해야 Exqutor 의 실험 결과를 제대로 해석할 수 있다.

4.1 해결하고자 하는 문제: 단조성(Monotonicity)의 부재

전통적인 데이터베이스 인덱스(B-tree 등)는 **단조성(monotonicity)**이라는 중요한 성질을 갖는다.

단조성이란? 인덱스를 탐색할 때, 다음에 방문할 데이터가 현재보다 "더 나쁜" 결과를 줄 것이라면, 탐색을 멈춰도 된다는 성질이다.

예를 들어, B-tree 에서 "가격 < 1000"인 상품을 찾을 때:

- 인덱스가 가격순으로 정렬되어 있으므로
- 가격이 1000 을 넘는 노드를 만나면 → 그 이후는 볼 필요 없음 → 탐색 종료

이 단조성 덕분에 B-tree 인덱스를 관계형 연산(필터링, 조인 등)과 효율적으로 결합할 수 있다.

그런데 **벡터 인덱스(HNSW 등)**에는 이 단조성이 없다. HNSW 그래프를 탐색할 때, 현재 방문한 노드의 거리가 커졌다고 해서 다음 노드도 멀 것이라는 보장이 없다. 그래프의 다른 경로를 따라가면 갑자기 가까운 벡터를 만날 수 있기 때문이다.

4.2 기존 접근법의 한계: TopK-only 인터페이스

단조성이 없기 때문에, 기존 벡터 검색 시스템들은 **TopK 인터페이스**만 제공한다:

- "가장 유사한 K 개를 찾아줘" → K 개의 결과를 반환

문제는 K 를 미리 정해야 한다는 것이다. 예를 들어:

- "유사한 상품을 찾아서, 그중 가격이 100 만원 이하인 것을 보여줘"

이 쿼리에서 K 를 얼마나 설정해야 할까? K=10 으로 하면 10 개 중 가격 조건을 만족하는 게 2 개뿐일 수 있고, K=10000 으로 하면 불필요한 벡터까지 검색하게 된다. 최적의 K 는 데이터에 따라 달라지므로 미리 알 수 없다.

기존 시스템들은 이 문제를 "**임시 단조성 인덱스(monotonicity-preserving tentative index)**"로 해결했다. 쿼리마다 TopK 결과를 구하고, 이를 임시 테이블로 만들어 관계형 연산과 결합하는 방식이다. 하지만 K 설정이 잘못되면 성능이 급격히 저하된다.

4.3 핵심 아이디어: 완화된 단조성 (Relaxed Monotonicity)

VBASE의 핵심 발견은 벡터 인덱스 탐색에도 **완화된 형태의 단조성**이 존재한다는 것이다.

완화된 단조성이란? 벡터 인덱스 탐색 과정에서, 탐색을 계속할수록 결과의 품질이 **통계적으로** 점차 나빠진다는 관찰이다. 엄격한 단조성처럼 "다음 결과가 반드시 더 나쁘다"는 보장은 없지만, 충분히 탐색하면 "더 이상 좋은 결과가 나올 확률이 매우 낮아진다"는 것은 보장된다.

이 성질을 이용하면:

1. TopK라는 고정 값 없이, 탐색 중 **동적으로** 종료 조건을 판단할 수 있다.
2. 벡터 인덱스를 관계형 연산자와 **직접** 결합할 수 있다 (임시 테이블 불필요).

4.4 Volcano 모델과의 통합

관계형 데이터베이스는 **Volcano 모델**(또는 Iterator 모델)이라는 표준 실행 모델을 사용한다:

- 각 연산자가 `Open()`, `Next()`, `Close()` 인터페이스를 제공
- `Next()`를 호출할 때마다 한 건씩 결과를 반환
- 연산자들이 트리 구조로 연결되어, 최하위 연산자부터 한 건씩 결과를 올려보냄

VBASE 는 벡터 인덱스를 이 Volcano 모델에 맞춰서:

1. `Open()`: 벡터 인덱스 탐색 초기화
2. `Next()`: 인덱스 내부 탐색을 한 단계 진행하고, 다음 후보 벡터를 반환
3. `Close()`: 탐색 종료

종료 조건으로 **완화된 단조성 체크**를 사용한다. "최근 N 번의 `Next()` 호출에서 조건을 만족하는 결과가 하나도 없으면 탐색 종료"와 같은 규칙이다.

4.5 지원하는 쿼리 유형

VBASE 는 다양한 복합 쿼리를 지원한다:

- **필터링된 유사도 검색:** 벡터 유사도 + 스칼라 조건 (WHERE distance < D AND price < 1000)
- **거리 임계값 필터링:** <-> 연산자로 범위 검색
- **멀티 벡터 쿼리:** approximate_sum() 함수로 여러 벡터 조건을 결합
- **벡터 조인:** 두 테이블 간 벡터 유사도 기반 조인

4.6 성능 결과

- 온라인 벡터 쿼리: 기존 벡터 시스템 대비 최대 **1,000 배** 성능 향상
- 분석적 유사도 쿼리: **7,000 배** 속도 향상, 99.9% 정확도
- Milvus 대비 200~300 배 낮은 쿼리 지연 시간 (96%+ 재현율)

4.7 본 논문과의 관계

VBASE 는 벡터 인덱스를 PostgreSQL 에 통합하면서, ANN 인덱스 접근을 **별도의 메모리 구조로 분리**하여 관리한다. 이로 인해 PostgreSQL 의 기본 버퍼 풀과는 독립적으로 동작한다. Exqutor 논문에서는 이 특성 때문에 "VBASE 에서는 정확한 비용 추정이 더 어렵다"고 지적한다. 동시에, VBASE 가 기본적으로 벡터 선택도를 50%로 고정 추정하기 때문에, Exqutor 의 ECQO 적용 시 가장 극적인 개선(최대 4 orders of magnitude)을 보인다.

논문 5. [11] Milvus: A Purpose-Built Vector Data Management System

저자: Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu 외 다수 (Zilliz, HUST)

학회/년도: SIGMOD 2021

분량: 약 14 페이지

이 논문의 의미

Milvus 는 **전문(특화) 벡터 데이터베이스의 대표 시스템**이다. Exqutor 가 대상으로 하는 "범용 벡터 DB"와 비교하면, 전문 벡터 DB 의 설계 철학과 한계를 이해할 수 있다. 또한 VBASE, SingleStore-V 실험에서 Milvus 가 비교 대상으로 자주 등장한다.

5.1 전문 벡터 DB 란 무엇인가

전문 벡터 DB 는 **벡터 데이터의 저장과 유사도 검색**에 특화된 시스템이다. 관계형 데이터베이스처럼 복잡한 SQL 쿼리를 지원하는 것이 목표가 아니라, 대규모 벡터를 빠르고 정확하게 검색하는 것이 목표이다.

Milvus 가 해결하는 핵심 도전 과제 5 가지:

1. **사용하기 쉬운 인터페이스**: 다양한 응용 분야의 개발자가 쉽게 벡터 검색을 활용
2. **이기종 컴퓨팅 최적화**: CPU 와 GPU 를 효율적으로 활용
3. **단순 유사도 검색을 넘어선 고급 쿼리**: 속성 필터링, 다중 벡터 검색 등
4. **동적 데이터 관리**: 실시간 삽입/삭제와 검색의 동시 처리
5. **확장성과 가용성**: 분산 환경에서의 대규모 서비스

5.2 시스템 구조

Milvus 는 **클라우드 네이티브 아키텍처**를 채택한다:

저장과 연산의 분리:

- 저장소: Amazon S3 같은 객체 스토리지를 사용하여 고가용성 확보
- 연산: 상태 없는(stateless) 컴포넌트들이 탄력적으로 확장/축소

세그먼트 기반 데이터 관리:

- 데이터는 **컬렉션(Collection)** 단위로 관리 (관계형 DB 의 테이블에 해당)
- 컬렉션은 여러 **세그먼트(Segment)**로 나뉨 (물리적 저장 단위)
- 세그먼트 단위로 인덱스를 구축하고 검색을 수행

조절 가능한 일관성(Tunable Consistency):

- 로그 백본(log backbone)을 활용하여 일관성-성능 트레이드오프를 사용자가 조절
- 강한 일관성(Strong): 쓰기 직후 읽기 보장, 하지만 느림
- 세션 일관성(Session): 같은 세션 내에서만 읽기 보장
- 최종 일관성(Eventual): 가장 빠르지만, 최신 데이터 반영이 늦을 수 있음

5.3 지원하는 벡터 인덱스

Milvus 는 다양한 시나리오에 맞는 벡터 인덱스를 지원한다:

인덱스 유형	특징	적합한 경우
FLAT	전수 검색 (brute-force)	소규모 데이터, 100% 정확도 필요 시
IVF_FLAT	클러스터링 기반 + 전수 검색	중규모 데이터, 높은 정확도
IVF_PQ	클러스터링 + 양자화(압축)	대규모 데이터, 메모리 절약
HNSW	계층적 그래프 기반	빠른 검색 필요, 메모리 충분
SCANN	Google 개발, 양자화 기반	높은 처리량 필요
DiskANN	SSD 기반	메모리 부족 시 대규모 데이터

5.4 고급 쿼리 기능

Milvus 는 단순 top-k 를 넘어서:

속성 필터링: 벡터 유사도 검색 시 스칼라 속성 조건을 함께 적용

```
results = collection.search(  
    data=[query_vector],  
    anns_field="embedding",  
    param={"metric_type": "L2", "params": {"nprobe": 10}},  
    limit=10,  
    expr="price < 100 AND category == 'electronics'" # 속성 필터  
)
```

다중 벡터 검색: 여러 벡터 필드를 동시에 검색하고 결과를 결합

5.5 실제 응용 사례

논문은 Milvus 위에 구축한 10 가지 실제 응용을 소개한다:

- 이미지/비디오 검색 시스템
- 화학 분자 구조 분석
- COVID-19 데이터셋 검색
- 개인화 추천 시스템
- 생체 인증 (다중 요소)
- 지능형 질의응답 (Q&A)

5.6 본 논문과의 관계: 전문 vs. 범용의 핵심 차이

Milvus 같은 전문 벡터 DB의 **근본적 한계**는 복잡한 관계형 연산(조인, 집계, 서브쿼리 등)을 지원하지 않는다는 것이다. 속성 필터링은 가능하지만, "여러 테이블을 조인하면서 벡터 검색도 수행하는" VAQ에는 적합하지 않다.

이것이 바로 Exqutor가 Milvus 대신 pgvector, VBASE, DuckDB 같은 **범용 벡터 DB**를 대상으로 하는 이유이다. 범용 벡터 DB는 SQL의 모든 기능을 지원하지만, 벡터 검색의 카디널리티 추정이 부정확하다는 문제가 있다. Exqutor는 이 문제를 해결한다.

논문 6. [20] Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL

저자: Yingzi Zhang, Shaolong Liu, Jianguo Wang (Purdue University)

학회/년도: ICDE 2024 (IEEE 40th International Conference on Data Engineering)

분량: 약 14 페이지

이 논문의 의미

이 논문은 pgvector 가 왜 전문 벡터 DB 에 비해 성능이 떨어지는지를 **근본 원인** 수준에서 분석한다.

Exqutor 가 pgvector 를 주요 실험 플랫폼으로 사용하므로, pgvector 의 내부 구조적 한계를 이해하면

Exqutor 의 실험 결과를 더 깊이 해석할 수 있다.

6.1 핵심 질문

"관계형 데이터베이스에서 벡터 데이터를 관리하는 데 **근본적인** 한계가 있는가, 아니면 단순히 **구현상의** 문제인가?"

이 질문에 답하기 위해, 저자들은 PostgreSQL 기반 벡터 확장(pgvector/PASE)의 소스 코드를 분석하고, Faiss(고성능 전문 벡터 라이브러리)와 성능을 비교한다.

6.2 발견한 핵심 한계점들

한계 1: 버퍼 풀(Buffer Pool) 오버헤드

PostgreSQL 은 모든 데이터 접근을 **공유 버퍼 풀(shared buffer pool)**을 통해 수행한다. 이것은 범용 데이터 관리에는 좋지만, 벡터 인덱스에는 심각한 오버헤드를 발생시킨다.

HNSW 그래프 탐색 시:

- 각 노드(벡터)를 방문할 때마다 버퍼 풀에서 해당 페이지를 찾아야 한다.
- 버퍼 풀 접근에는 **래치(latch)** 획득이 필요하다 (동시성 제어를 위해).
- HNSW 는 탐색 중 많은 노드를 무작위로 방문하므로, 래치 획득/해제가 매우 빈번하게 발생한다.
- 이 래치 오버헤드가 전체 검색 시간의 상당 부분을 차지한다.

Faiss 는 벡터를 메모리에 직접 배치하여, 이런 오버헤드 없이 빠르게 접근한다.

한계 2: 튜플 접근(Tuple Access) 오버헤드

PostgreSQL 에서 데이터를 읽으려면:

1. 페이지를 버퍼 풀에서 찾고
2. 페이지 내에서 튜플(행) 오프셋을 계산하고
3. 튜플의 헤더(트랜잭션 정보, 가시성 정보 등)를 파싱하고
4. 실제 데이터(벡터)를 추출한다

이 과정에서 **튜플 헤더 파싱**과 **가시성 확인(MVCC)**이 벡터 검색에 불필요한 오버헤드를 추가한다. Faiss 는 벡터만 저장하므로 이런 부담이 없다.

한계 3: 공간 증폭(Space Amplification)

HNSW 인덱스가 PostgreSQL 에서 구축되면, **기본 테이블의 크기를 훨씬 초과**하는 공간을 사용한다.

이유:

- PostgreSQL 의 페이지 구조(8KB 고정 블록 + 헤더)에 맞춰야 하므로 패딩(낭비 공간)이 발생
- HNSW 그래프의 연결(edge) 정보도 페이지 단위로 저장되어 비효율적
- MVCC 를 위한 추가 메타데이터(xmin, xmax 등)가 각 벡터와 함께 저장

이로 인해 인덱스가 메모리에 올라가지 못하면 **디스크 I/O** 가 발생하여 성능이 급격히 저하된다.

한계 4: 인덱스 구축 시간

PostgreSQL 에서 HNSW 인덱스를 구축하는 것은 Faiss 대비 **수배~수십배** 느리다.

원인:

- 벡터를 삽입할 때마다 버퍼 풀을 경유해야 한다.
- 인덱스 구축 중 WAL(Write-Ahead Log) 로깅이 발생한다.
- `maintenance_work_mem` 제한으로 인해, 대규모 인덱스 구축 시 중간 데이터가 디스크로 스푼(spill)된다.

한계 5: 쿼리 최적화의 한계

PostgreSQL 의 옵티마이저는 벡터 데이터에 대한 **통계 정보를 수집하지 않는다**. 기존의 히스토그램 기반 통계는 스칼라 값에 적합하지만, 고차원 벡터의 분포를 표현하기에는 부적절하다.

이로 인해:

- 벡터 유사도 조건의 선택도를 **고정 상수**로 추정한다 (pgvector 의 경우 33.3%).
- 조인 순서, 조인 방식 등의 결정이 부정확해진다.

6.3 근본적인가, 구현상의 문제인가?

논문의 결론은 일부는 근본적이고, 일부는 구현 개선으로 해결 가능하다는 것이다:

근본적 한계:

- 버퍼 풀과 MVCC 는 PostgreSQL 의 핵심 설계 원칙이다. 이를 제거하면 PostgreSQL 이 아니게 된다.
- 범용 관계형 DB 의 페이지 기반 저장 구조는 벡터 인덱스의 무작위 접근 패턴과 근본적으로 맞지 않는다.

구현 개선 가능:

- 벡터 전용 버퍼 풀을 추가하여 래치 오버헤드를 줄일 수 있다 (VBASE 가 이 접근법을 사용).
- 벡터 통계 수집 기능을 추가할 수 있다 (Exquitor 가 이 부분을 해결).
- 인덱스 구축 최적화(병렬 구축, 메모리 관리 개선)는 구현 수준에서 해결 가능.

6.4 본 논문과의 관계

이 논문이 발견한 "PostgreSQL 의 옵티마이저가 벡터 통계를 수집하지 않아 고정 선택도를 사용한다"는 문제가 바로 **Exqutor 가 해결하는 핵심 문제**이다.

Exqutor 의 ECQO 는 통계 정보 없이도, 벡터 인덱스를 직접 탐색하여 정확한 카디널리티를 얻는다. 또한 이 논문이 언급한 "공간 증폭" 문제는 Exqutor 논문에서도 직접 인용되며, PostgreSQL 비용 모델이 HNSW 인덱스의 실제 비용을 과대추정하는 원인으로 지목된다.

6 편 요약 비교표

논문	핵심 키워드	본 논문과의 관계	난이도
[1] AnalyticDB-V	하이브리드 쿼리, ANNS 연산자, 비용 기반 최적화	VAQ 개념의 원조. 카디널리티 추정은 미해결	★★★★☆☆
[10] VBASE	완화된 단조성, Volcano 모델, 벡터+관계형 통합	실험 대상 시스템. 가장 극적인 개선(10,000 배)	★★★★★☆☆
[7] SingleStore-V	Top() 연산자, 세그먼트 인덱스, 플러그형 인덱스	경쟁 접근법. top-k 특화 vs. Exqutor 의 조인 최적화	★★★★☆☆☆
[11] Milvus	전문 벡터 DB, 클라우드 네이티브, 세그먼트 관리	전문 vs. 범용 벡터 DB 의 차이를 보여줌	★★★☆☆☆☆
[6] Context-Enhanced Joins	E-연산자, 텐서 조인, 인덱스 vs. 스캔 분석	이론적 기반. 정확한 카디널리티가 왜 중요한지	★★★★★☆☆
[20] PostgreSQL 한계	버퍼 풀, 공간 증폭, 고정 선택도	Exqutor 가 해결하는 문제의 근본 원인 분석	★★★★☆☆☆